

Tier 1: Fire and Smoke Detection



What problem am I solving?

This project addresses the recognition of wildfire and smoke in nature, improving response times. Agriculture enterprises, national reserves, and wildlife refuges are primarily affected by wildfires. In 2024, more than 56,000 wildfires across the country were reported. It is important to address this problem to preserve nature and safety.



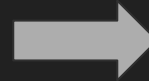
Fire and Smoke real-time recognition system

The system will detect active wildfires and smoke in real-time using image analysis.

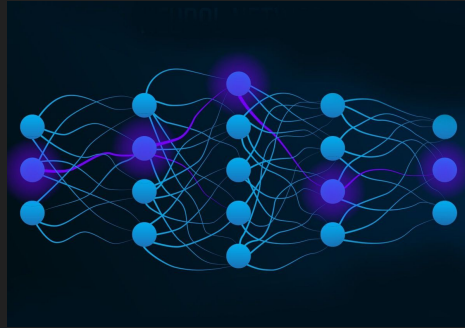
The system will receive image input from security cameras, drones, etc.



YOLO detects fire/smoke.



If there is fire/smoke, the system will print a message letting know the responsables.



Technical Approach

CV technique: Object detection with YOLO.

Model: YOLOv8.

Frameworks: Pytorch and Ultralytics.

Why this approach?

I decided to use YOLOv8 because it has a high rate of accuracy measured by Microsoft COCO and Roboflow 100; and it has a large community, meaning that it has many online guides and tutorials.

DATA PLAN

To train my model I will use a public dataset from kaggle:

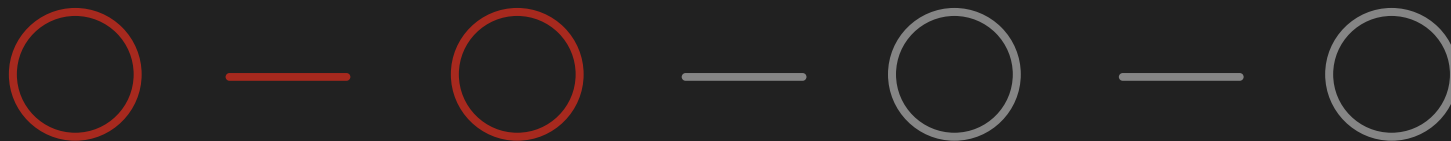
<https://www.kaggle.com/datasets/elmadafri/the-wildfire-dataset>

This dataset is very complete and varied. It contains day and night images, different angles, different species of trees and plants, some have hills, etc. Since the variability in this dataset is really good I will not need synthetic data.

This dataset is already labeled and contains around 2700 images.



System Diagram



Input

The system will receive images from user, security cameras, drones, etc.

Preprocessing

Resize, normalize and convert the images to tensors, ready for the model to analyze them.

YOLOv8 Prediction

The model will receive the images already processed and transformed to tensors. It will use the learnt important features and hidden patterns to determine if there is or not an active wildfire.

Output

<20% probabilities of being a wildfire: the system will not send any type of message or advice.

21% to 50%: the system will send a Yellow alert, suggesting the responsables to check manually.

51% - 80%: an Orange Alert will be sent to the security staff.

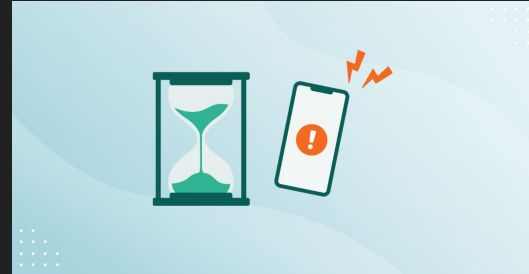
81% - 100% : Red Alert, urgent notification send to security staff and Firefighters.

Success Metrics

Primary metric: Recall, which is a good indicator of the ability of the model to identify the positive class (in this case, the presence of an active wildfire). Since this is a prevention work, above an 80% recall would perfectly work.

Secondary metric: Response Time. The goal would be to achieve a response time of 1 second or less, this will allow the model to perform real-time analysis.

$$\text{Recall} = \frac{\text{True Positives}}{\text{True Positives} + \text{False Negatives}}$$

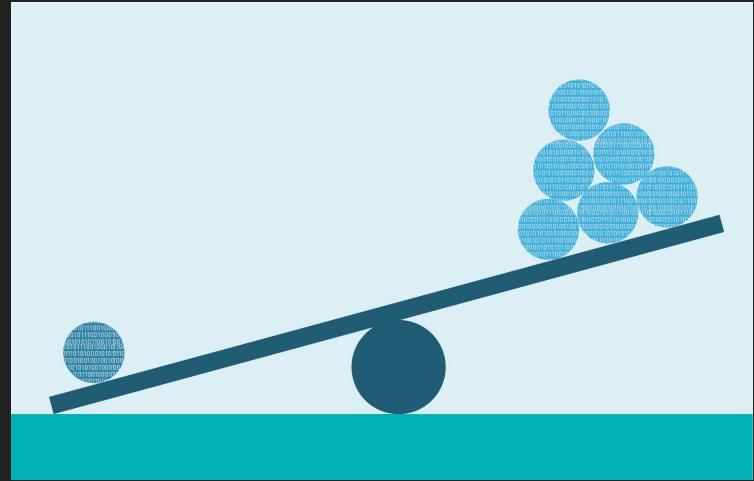


Challenges and backup plans

Data Imbalance: There is a little imbalance in my dataset, since the difference is small there should be no problem. But in case of bias due to data imbalance I will use synthetic data (rotation, scaling, saturation, etc.) to balance the dataset.

Latency: the model has slow response times. The backup plan is trying to change the architecture or even the model selected to achieve a better time performance.

Adversarial conditions: some weather conditions can create noise that will affect the model performance. Add new data to teach the model how to perform in extreme weather conditions.



Week By Week plan

For this project I designed a four weeks plan:

1st Week: Set up the environment and the preprocessing of images.

2nd week: Apply and train the YOLOv8 model.

3rd week: Test the model and optimize parameters to improve its performance.

4th week: Use gradio to make an interface. Supervise the final system.



Resources needed

Compute: Google Colab and Kaggle

Frameworks: PyTorch, Ultralytics and Gradio.

Estimated cost: 0\$

